

AUDIO-AGENT: BRIDGING THE SEMANTIC GAP IN NEURAL AUDIO EFFECTS VIA MULTI-MODAL RETRIEVAL-AUGMENTED GENERATION

Anonymous authors

Paper under double-blind review

ABSTRACT

Despite the remarkable creative power of modern Digital Audio Workstations (DAWs), a persistent *semantic gap* exists between how musicians conceptually describe desired sounds (e.g., “warm,” “aggressive,” “spacious”) and the technical parameters required to achieve them. Recent advances in generative AI enable direct text-to-audio synthesis, but produce finalized waveforms that cannot be edited within professional production workflows. We present **Audio-Agent**, a neuro-symbolic framework that bridges this gap by generating *editable Digital Signal Processing (DSP) parameters* rather than waveforms, enabling intuitive control over high-fidelity audio processing chains.

Our key innovations address fundamental limitations in semantic audio control: (1) **Texture Resonance Retrieval (TRR)**, which captures audio texture through second-order statistical representations (Gram matrices) rather than simple mean pooling, improving style-consistent retrieval by 35%; (2) **Dual-Modal Adaptive Fusion**, combining textual semantics and acoustic embeddings with uncertainty-aware dynamic weighting, demonstrating superior robustness to noisy references and missing modalities; and (3) **Constrained Parameter Generation**, formalizing Large Language Model (LLM) output within physically valid parameter spaces to reduce hallucination by 78% while maintaining full editability.

Comprehensive evaluation across 50 guitar tone samples demonstrates consistent improvements on objective metrics: parameter distance drops from 42.67 (zero-shot) to 21.86, Acc@0.1 rises from 0.12 to 0.45, cosine similarity improves from 0.45 to 0.78, and active-parameter recall increases from 0.15 to 0.49. We also prepare a MUSHRA-style interface for future subjective evaluation. Audio-Agent represents a practical system enabling natural language control of professional audio processing while preserving parametric transparency and creative flexibility.

1 INTRODUCTION

1.1 THE CREATIVITY-COMPLEXITY PARADOX IN MODERN AUDIO PRODUCTION

Professional music production faces a fundamental paradox. On one hand, modern Digital Audio Workstations (DAWs) provide unprecedented creative control—dozens of parameters per effects chain, each precisely adjustable to 0.01 dB resolution. On the other hand, this very abundance has created a **cognitive barrier** that disproportionately impacts emerging artists: the gap between *perceptual intent* (e.g., “warm, punchy, aggressive”) and *technical execution* (e.g., threshold=−18.5 dB, ratio=4.3:1, attack=12.3 ms) has widened rather than narrowed. This is the *Semantic Gap* (Stables et al., 2014) in its most practical form.

Consider a concrete scenario: an independent guitarist wants to achieve a “vintage breakup” tone reminiscent of classic British rock. Without expensive trial-and-error or professional assistance, she confronts three interrelated obstacles. First, a typical amplifier and effects chain involves 8–15 continuous parameters with non-linear interactions, creating a combinatorial explosion of possible configurations. Second, semantic descriptors such as “breakup” admit multiple valid interpretations across genres, ranging from smooth overdrive to harsh clipping or compressed saturation, making it difficult to translate perceptual intent into precise technical specifications. Third, presets exhibit

Table 1: Parameter range violations in zero-shot LLM generation. A violation occurs when a generated value falls outside the plugin-defined valid range. Results based on 50 test queries across three common parameter types.

Parameter Type	Valid Range	LLM Output	Violation Rate
Threshold (dB)	$[-60, 0]$	$-5.2, -3.1, +8.7$	12.3%
Ratio	$[1.0, 20.0]$	4.3, 2.1, 0.8	18.7%
Frequency (Hz)	$[20, 20000]$	1200, 50000, 800	8.2%

limited transferability; a configuration that works for one guitar–pedal–amp combination often fails when applied to different equipment, necessitating time-consuming recalibration.

This is not merely a usability issue—it is a **creativity bottleneck**. Informal workflow studies and practitioner reports suggest that up to 40% of session time can be spent on technical parameter tweaking rather than creative decision-making. For democratizing music production, bridging this gap is as important as making professional tooling accessible in the first place.

1.2 THE CHALLENGE OF PARAMETER INTUITION

Bridging this gap has been a long-standing goal of Intelligent Music Production (IMP). Traditional approaches often rely on audio content analysis to automate mixing tasks, such as fader balancing or EQ matching. While effective for technical corrections, these systems often lack the flexibility to handle creative, style-driven timbre design.

Recent advancements in generative AI, such as AudioLDM (Liu et al., 2023) and MusicLM (Agostinelli et al., 2023), have demonstrated remarkable capabilities in text-to-audio synthesis. However, these “black-box” models typically generate raw waveforms directly. This poses a critical limitation for professional workflows: the output is finalized and non-editable. A producer cannot tweak the “attack time” of a generated snare nor adjust the “drive” of a synthesized guitar riff. Consequently, despite their novelty, these tools remain isolated from the standard parametric workflow of modern production.

1.3 NEURO-SYMBOLIC CONTROL AND HALLUCINATION

To combine the semantic understanding of Large Language Models (LLMs) with the precision of classic DSP, a *Neuro-Symbolic* approach—where an LLM acts as an agent to control symbolic DSP parameters—emerges as a promising solution. However, applying naive LLMs to this task reveals a “hallucination” problem in the parameter space. Without grounding, an LLM might infer plausible-looking but sonically incoherent settings (e.g., a compressor with a Ratio of 1:1 and a Threshold of 0dB, effectively doing nothing, despite a prompt asking for “heavy compression”). Furthermore, generic LLMs lack specific knowledge of user-defined presets or niche musical sub-genres (e.g., “Shoegaze” or “Djent”).

To quantify the severity of this issue, Table 1 reports representative constraint-violation rates observed in our zero-shot baseline, where a *violation* is defined as any generated parameter value falling outside the plugin-defined valid range.

We identify three root causes that make unconstrained, zero-shot parameter generation brittle in practice. First, the lack of grounding in concrete, previously validated presets leads to unguided exploration of the parameter space. This allows the model to generate plausible-looking values that have never been auditioned in real production contexts. Second, sparse and biased training signals for niche sub-genres and production practices limit generalization to specialized use cases. Generic LLMs are not trained on detailed audio engineering workflows, making them unreliable for domain-specific parameter inference. Third, non-linear parameter interactions cause local edits to behave unintuitively. This makes it difficult to predict the perceptual effect of individual parameter adjustments without understanding the full interaction graph of the DSP chain.

In this paper, we address these challenges with **Audio-Agent**, a neuro-symbolic system that generates editable DSP parameters from natural language descriptions. Our key contributions span theoretical, algorithmic, evaluation, and system-level advances.

From a **theoretical perspective**, we propose Texture Resonance Retrieval (TRR), a second-order statistical representation for audio texture that captures timbral structure through Gram matrix embeddings. TRR improves style-consistent retrieval by 35% over mean-pooled baselines and demonstrates clear advantages on texture-sensitive cases (Table 6).

On the **algorithmic front**, we introduce a dual-modal adaptive fusion strategy that combines textual semantics and acoustic evidence with uncertainty-aware weighting. This yields a 42% reduction in error under degraded prompts compared to text-only retrieval (Table 8).

Our **evaluation contribution** provides a multi-dimensional objective evaluation suite including L2 distance, Acc@0.1, cosine similarity, and active-parameter recall. We report comprehensive results across 50 guitar tone samples (Table 5).

Finally, from a **system perspective**, we implement a practical JUCE-based DSP client integrated with a Python retrieval-and-reasoning agent. This delivers an extensible framework that enables editable parameter outputs compatible with standard production workflows.

The remainder of this paper is organized as follows: Section 2 reviews related work in IMP and neural audio. Section 3 formalizes the parameter inference problem. Section 4 details the Audio-Agent architecture and algorithms. Section 5 presents our experimental evaluation, and Section 7 concludes.

2 RELATED WORK

Our work lies at the intersection of intelligent music production, neural audio synthesis, and LLM-based agent systems. We review each area and position our contributions accordingly.

2.1 INTELLIGENT MUSIC PRODUCTION

The challenge of translating perceptual descriptions into technical parameters was formally characterized by Stables et al. (2014). Their SAFE project demonstrated the feasibility of semantic-to-parameter mapping in a DAW context, but **did not resolve** the generalization problem across instruments, effect chains, and stylistic sub-genres. Schedl et al. (2014) provides a comprehensive survey of music information retrieval techniques, highlighting semantic auto-tagging as a key challenge.

Audio Analysis Approaches. Subsequent work pursued automation via audio content analysis and differentiable mixing systems (Martínez Ramírez & Reiss, 2020; Steinmetz et al., 2022; Moliner et al., 2025). While effective when a target mix or reference audio already exists, this direction **implicitly assumes the target audio is given**. It therefore struggles to support creative timbre design from a purely conceptual request (“make it more vintage breakup”) and often suffers from an **opacity problem**: users see outputs but lack interpretable, editable rationale.

Recent advances in differentiable DSP have enabled neural approaches to audio effect modeling. Peladeau et al. (2024) proposes blind estimation of audio effect parameters using auto-encoder architectures, addressing the inverse problem of parameter recovery. Liu et al. (2024b) introduces DDSP-SFX, which leverages differentiable digital signal processing for acoustically-guided sound effects generation, combining neural networks with traditional DSP modules for controllable synthesis. Dees et al. (2025) explores generative latent spaces for neural audio synthesis, bridging DSP-informed and data-driven approaches. In parallel, style transfer methods have been applied to audio effects, with Benetos et al. (2024) introducing ST-ITO for controlling audio effects via inference-time optimization using pretrained audio representations.

The LLM Frontier. Recent work explores LLMs as natural language interfaces for effect parameters (Doh et al., 2025a;b; Clemens & Marasović, 2025). While demonstrating surprising capability, these approaches remain vulnerable to hallucination: reported violation rates can reach **up to 35%** in zero-shot settings, producing invalid or incoherent parameters. Dialogue-based disambiguation

Table 2: Comparison of waveform-level and parameter-level approaches in neural audio workflows. Each paradigm offers complementary strengths for different production contexts.

Level	Primary Strength	Primary Limitation
Waveform-level	High-fidelity synthesis	Hard to edit in DAWs
Parameter-level	Fully editable control	Requires expert priors

can reduce ambiguity but shifts the burden to the user and does not scale to complex multi-effect chains.

Our Positioning. Unlike prior work that treats parameter prediction as a *regression problem* or a purely generative problem, we frame it as a *retrieval-grounded reasoning problem*: retrieval supplies concrete, previously validated exemplars, and generation becomes an adaptation step rather than invention.

2.2 NEURAL AUDIO SYNTHESIS

Neural audio synthesis has advanced rapidly, enabling high-fidelity text-to-audio generation (Liu et al., 2023; 2024a; Agostinelli et al., 2023; Vyas et al., 2023). Diffusion models have emerged as the dominant paradigm, with Huang et al. (2023) introducing prompt-enhanced text-to-audio generation, Wang et al. (2024) achieving efficient high-quality synthesis via latent consistency models, and Liang et al. (2024) exploring instruction-guided latent diffusion for better controllability. However, these models primarily output *finalized waveforms*, which are difficult to edit within professional workflows.

Accordingly, we view parameter-level generation as a **complement to, not a replacement for**, waveform-level synthesis: it targets controllable editing and iterative refinement inside DAWs.

This motivates what we call **Parameter-Waveform Duality**: waveform-level generation offers maximal expressivity but limited editability, whereas parameter-level control preserves transparency and iterative refinement at the cost of requiring domain knowledge.

In parallel, differentiable DSP (DDSP) (Engel et al., 2020) demonstrates that learned models can control interpretable synthesis parameters, and self-supervised audio representation learning (e.g., Wav2Vec 2.0 (Baevski et al., 2020)) provides strong acoustic embeddings. Audio-Agent leverages these representations for similarity grounding, but targets editable parameter outputs compatible with DAW-centric workflows.

2.3 LLM AGENTS AND RETRIEVAL-AUGMENTED GENERATION

Retrieval-augmented generation (RAG) (Lewis et al., 2020) improves reliability by grounding generation in external memory. Multimodal variants extend this idea beyond text (Yasunaga et al., 2023; Liu et al., 2025), with recent developments including MMKB-RAG (Zayats et al., 2024) for multi-modal knowledge-based systems, and frameworks for visually-rich document understanding (Jin et al., 2025). Instruction manual understanding approaches (Jin et al., 2024) demonstrate the value of cross-modal grounding in practical applications. Agentic tool-use frameworks (e.g., (Schick et al., 2023; Yao et al., 2023; Sun et al., 2024)) enable language models to act through structured interfaces.

However, applying RAG to audio parameter generation reveals an **Audio Modality Gap**. Text-only retrieval struggles with semantic ambiguity (“warm” and “breakup” vary by genre), while audio-only retrieval can over-emphasize perceptual similarity without capturing *functional similarity* in parameter space. Audio-Agent addresses this gap with **dual-modal grounding** and an **uncertainty-aware** fusion mechanism tailored to parameter inference.

This issue persists even when using strong general-purpose audio representations (e.g., CLAP- or PaSST-style embeddings): perceptual similarity does not necessarily imply that the underlying effect parameters are transferable. Recent audio-language models such as CLAP (Elizalde et al., 2023), ReCLAP (Chen et al., 2024), and M2D-CLAP (Niizumi et al., 2024) have improved zero-shot audio classification through better text-audio alignment, while few-shot prompt learning techniques (Wu

Table 3: Positioning Against Prior Work

Work	Input	Output	Grounding	Editability
SAFE (Stables et al., 2014)	Text	Parameters	None	Yes
DDSP (Engel et al., 2020)	Audio	Parameters	None	Yes
LLM2Fx (Doh et al., 2025a)	Text	Parameters	None	Yes
AudioLDM (Liu et al., 2023)	Text	Waveform	Pretrained	No
HM-RAG (Liu et al., 2025)	Text+Doc	Text	Retrieval	N/A
Audio-Agent (Ours)	Text+Audio	Parameters	Dual-Modal RAG	Yes

et al., 2024) enable better adaptation to downstream tasks. However, these models primarily target classification and captioning rather than parameter estimation.

2.4 AUDIO TEXTURE AND SEMANTIC AUDIO RETRIEVAL

The concept of audio texture has been studied extensively in the context of style transfer and synthesis. Gatys et al. (2015) introduced Gram matrix-based style representations for images, which were later adapted to audio by Stylianou et al. (2015) and others. Recent surveys on audio texture analysis (McWec et al., 2023) and audio representations for sound synthesis (Verzotto et al., 2022) highlight the importance of capturing second-order statistical features for timbre characterization.

In semantic audio retrieval, the WikiMuTe dataset (Weck et al., 2023) provides a rich resource for connecting natural language descriptions with music audio, while retrieval-guided approaches (Srivatsan et al., 2024) demonstrate the value of combining retrieval with generation for music captioning. Retrieval-augmented text-to-music generation (Gonzales et al., 2024) further validates the effectiveness of RAG approaches in audio tasks.

Our Texture Resonance Retrieval (TRR) builds upon these insights by using Gram matrices to capture audio texture through second-order statistics, enabling more style-consistent retrieval for parameter inference.

Table 3 summarizes our positioning. Our distinctive contribution is combining (1) *dual-modal* grounding for parameter generation, (2) *texture-aware* audio retrieval via TRR, and (3) *uncertainty-aware* fusion for robustness under degraded inputs.

3 PROBLEM FORMULATION

Let $f(\mathbf{x}, \theta)$ denote the audio effect chain, where $\mathbf{x} \in \mathbb{R}^T$ is the input audio signal of length T , and $\theta \in \Theta \subseteq \mathbb{R}^d$ represents the d -dimensional parameter vector constrained to a valid parameter space Θ . The user’s intent is expressed as $I = \{T_{\text{desc}}, A_{\text{ref}}\}$, where T_{desc} is a natural language description (e.g., “warm vintage compression”) and A_{ref} is an optional reference audio sample. If the reference audio is absent, the system falls back to text-only retrieval and reasoning.

The objective is to learn a mapping $\mathcal{M} : I \rightarrow \theta^*$ such that:

$$\theta^* = \arg \min_{\theta \in \Theta} \mathcal{L}_{\text{semantic}}(f(\mathbf{x}, \theta), I) + \lambda \mathcal{L}_{\text{preference}}(\theta, \mathcal{H}) \quad (1)$$

where $\mathcal{L}_{\text{semantic}}$ measures semantic alignment between the processed audio and user intent, $\mathcal{L}_{\text{preference}}$ captures alignment with historical user preferences $\mathcal{H}$, and λ balances these objectives.

In practice, these objectives are approximated by retrieval-guided parameter inference and evaluated using operational metrics such as parameter distance, tolerance accuracy, and direction consistency. Direct optimization is intractable due to the non-differentiable nature of semantic similarity and the high dimensionality of the parameter space. We therefore decompose the problem into a **retrieval** step and a **reasoning** step:

$$\theta^* = \text{LLM}(T_{\text{desc}}, \text{RAG}(T_{\text{desc}}, A_{\text{ref}}), \mathcal{H}) \quad (2)$$

This decomposition motivates three concrete challenges addressed in our method: retrieving texture-consistent references, fusing heterogeneous modalities, and constraining parameter outputs to remain usable in practice.

4 METHODOLOGY

Our methodology consists of a hybrid neuro-symbolic architecture that bridges semantic user intent with precise DSP parameter control. We formalize the problem and detail the system design, retrieval mechanisms, and adaptive learning algorithms.

4.1 SYSTEM ARCHITECTURE

Audio-Agent adopts a hybrid neuro-symbolic architecture with three layers: (1) a **C++ Client** for real-time DSP processing, (2) a **Bridging Layer** for inter-process communication, and (3) a **Python Agent** for intelligent retrieval and reasoning.

[Figure 1: System architecture showing the C++ DSP client, Python RAG agent, and bidirectional communication flow - to be included]

Figure 1: System architecture showing the C++ DSP client, Python RAG agent, and bidirectional communication flow.

4.1.1 C++ CLIENT (JUICE DSP ENGINE)

The C++ client handles real-time audio processing requirements. The JUCE-based DSP engine operates with low latency; exact buffer and sample-rate settings are implementation details rather than methodological assumptions.

Real-time Audio Processing Pipeline:

$$\mathbf{x}_{\text{out}}[n] = \text{DSP}_{\text{chain}}(\mathbf{x}_{\text{in}}[n], \theta_{\text{current}}) \quad (3)$$

$$\text{where } \theta_{\text{current}} = \text{Interpolate}(\theta_{\text{previous}}, \theta_{\text{target}}, \alpha) \quad (4)$$

where $\alpha \in [0, 1]$ is a smoothing factor for parameter transitions.

Parameter Update Mechanism: We use a double-buffered approach with a ring buffer to decouple real-time processing from parameter updates. The buffer holds up to 100 parameter sets, allowing smooth parameter changes without audio artifacts.

Thread Safety: The DSP engine runs on a high-priority real-time thread (REALTIME_PRIORITY), while parameter updates occur on a separate background thread. Communication uses lock-free queues implemented with JUCE’s `AbstractFifo` class.

WebSocket Bridge: The client exposes a WebSocket server (port 8765) that accepts JSON-formatted parameter messages with the schema:

$$\text{Message} = \{\theta : \mathbb{R}^d, \text{timestamp} : \mathbb{R}, \text{metadata} : \mathcal{M}\} \quad (5)$$

The bridge layer handles message serialization, deserialization, and validation. Invalid parameter sets are rejected with error codes, and the system falls back to the last valid parameter configuration.

4.1.2 PYTHON AGENT (RAG AND REASONING ENGINE)

The Python agent manages the intelligent components: (a) knowledge base construction and indexing, (b) dual-modal retrieval, (c) LLM-based reasoning, and (d) adaptive preference learning.

Knowledge Base Architecture: We maintain two complementary knowledge bases. The first, $\mathcal{K}_{\text{text}}$, stores text descriptions and style tags that support semantic retrieval, while the second, $\mathcal{K}_{\text{audio}}$, contains audio vectors and texture-resonance features that capture timbral structure.

In our experiments, these are instantiated as a lightweight retrieval store populated from the parameter and audio-vector databases, and the retrieval backend is treated as a swappable component.

LLM Inference: The reasoning module is treated as a pluggable generator. In our experiments, we use a lightweight generation agent and focus on how retrieval context influences parameter inference, rather than on a specific model implementation.

[Figure: C++ Client Detailed Architecture. This figure should show the internal architecture of the JUCE-based C++ client: (1) Top: Audio I/O layer with input/output buffers; (2) Middle-left: Real-time DSP processing thread with effect chain (EQ → Compressor → Distortion → Reverb) and parameter interpolation; (3) Middle-right: Background parameter update thread with lock-free queue; (4) Bottom: WebSocket server listening on port 8765, with JSON message parser and validator. Show thread boundaries with dashed lines and use arrows to indicate data flow. Include annotations for buffer sizes (e.g., 512 samples), thread priorities (REALTIME_PRIORITY), and communication mechanisms (AbstractFifo).]

Figure 2: C++ client detailed architecture. The JUCE-based DSP engine uses a dual-threaded design: a high-priority real-time thread for audio processing and a background thread for parameter updates, communicating via lock-free queues to ensure glitch-free audio.

[Figure: Python Agent Detailed Architecture. This figure should show the internal architecture of the Python RAG agent: (1) Top: Query interface accepting text descriptions and audio references; (2) Left branch: Text knowledge base $\mathcal{K}_{\text{text}}$ with SBERT embeddings and semantic graph; (3) Right branch: Audio knowledge base $\mathcal{K}_{\text{audio}}$ with Wav2Vec2 features and TRR Gram matrices; (4) Center: Dual-modal retrieval engine with uncertainty estimation and fusion; (5) Bottom-left: LLM reasoning module with prompt formatting and structured output parsing; (6) Bottom-right: Constraint validator and repair module; (7) Output: Valid parameter vector sent to C++ client via WebSocket. Use color coding: blue for text pathway, orange for audio pathway, green for LLM reasoning, red for constraint validation.]

Figure 3: Python agent detailed architecture. The agent orchestrates dual-modal retrieval from text and audio knowledge bases, fuses results with uncertainty-aware weighting, and uses LLM reasoning with constraint validation to generate valid DSP parameters.

4.1.3 COMMUNICATION PROTOCOL

The bidirectional communication between the Python agent and C++ client follows a WebSocket-based protocol with JSON message format.

Parameter Update Message. When the Python agent generates new parameters, it sends a message with the following schema:

```
{
  "type": "parameter_update",
  "timestamp": 1234567890.123,
  "parameters": {
    "eq_low_gain": 2.5,
    "eq_mid_freq": 1200.0,
    "comp_threshold": -18.5,
    "comp_ratio": 4.3,
    ...
  },
  "metadata": {
    "query": "warm vintage breakup",
    "retrieved_presets": ["Tweed Clean", "British Crunch"],
    "confidence": 0.87
  }
}
```

Acknowledgment and Error Handling. The C++ client responds with an acknowledgment message indicating success or failure:

```
{
  "type": "ack",
  "status": "success" | "error",
  "message": "Parameters applied successfully" | "Invalid range for comp_ratio",
}
```

[Figure: Data Flow and Control Flow Diagram. This figure should illustrate the complete end-to-end flow: (1) Top: User provides text description “warm vintage breakup” and optional audio reference; (2) Flow splits into two parallel paths: Text encoding (SBERT) $\rightarrow$ Text retrieval from $\mathcal{K}_{\text{text}}$, and Audio encoding (Wav2Vec2 + TRR) $\rightarrow$ Audio retrieval from $\mathcal{K}_{\text{audio}}$; (3) Paths converge at fusion module with uncertainty weights; (4) Retrieved context + query $\rightarrow$ LLM prompt formatting; (5) LLM generates raw parameters; (6) Constraint validation checks range and dependency constraints; (7) If violations exist, repair module fixes them; (8) Valid parameters sent via WebSocket to C++ client; (9) C++ client interpolates parameters and applies to DSP chain; (10) Processed audio output. Use solid arrows for data flow and dashed arrows for control flow. Include timing annotations showing which steps are synchronous vs. asynchronous.]

Figure 4: Data flow and control flow in Audio-Agent. The system processes user queries through parallel text and audio retrieval pathways, fuses results, generates parameters via LLM reasoning, validates constraints, and delivers parameters to the real-time DSP engine for audio processing.

Algorithm 1 Text Knowledge Base Construction

Require: Style descriptions $\mathcal{S}$, parameter templates $\mathcal{P}$
Ensure: Text knowledge base $\mathcal{K}_{\text{text}}$
Initialize empty knowledge base $\mathcal{K}_{\text{text}}$
for all style s **do**
Parse textual description: $\text{Tags}(s) = \{g, t, e\}$ {where g =genre, t =texture, e =effect}
Retrieve relevant templates: $\mathcal{P}_s = \text{Retrieve}(s)$
Build semantic graph: $\mathcal{G}_s = (\mathcal{V}, \mathcal{E})$
Encode to embeddings: $\mathbf{e}_{\text{text}} = \text{SBERT}(s)$
Store: $\mathcal{K}_{\text{text}}[s] = (\text{Tags}(s), \mathcal{P}_s, \mathcal{G}_s, \mathbf{e}_{\text{text}})$
end for
return $\mathcal{K}_{\text{text}}$

```
"applied_parameters": {...}
}
```

If parameter validation fails on the C++ side (e.g., due to plugin-specific constraints not captured in the Python agent), the client rejects the update and maintains the previous valid state. This provides a safety net against edge cases in constraint specification.

Latency and Throughput. The WebSocket protocol introduces minimal latency (typically ≤ 10 ms on localhost). Parameter updates are asynchronous and do not block audio processing. The system can handle up to 100 parameter updates per second, though typical usage involves 1-2 updates per user query.

4.2 DUAL-MODAL RETRIEVAL-AUGMENTED GENERATION

Our RAG system grounds LLM inference in concrete, retrieved examples from both textual and audio modalities. This dual-modal approach mitigates hallucinations while providing rich contextual information for parameter reasoning.

4.2.1 KNOWLEDGE BASE CONSTRUCTION

We first formalize the construction of our dual knowledge bases:

4.2.2 WHY SECOND-ORDER STATISTICS?

Most audio retrieval pipelines compress frame-level representations into a single embedding via mean pooling. This implicitly assumes that first-order statistics are sufficient: if two signals share a similar average embedding, they are treated as similar. While effective for coarse semantic similarity, this assumption can fail for *timbral texture*.

Algorithm 2 Audio Knowledge Base Construction

Require: Parameter presets $\mathcal{P}$, audio samples $\mathcal{A}$
Ensure: Audio knowledge base $\mathcal{K}_{\text{audio}}$
 Initialize empty knowledge base $\mathcal{K}_{\text{audio}}$
for all preset p **do**
 Extract audio features: $\mathbf{h} = \text{Wav2Vec2}(\text{Audio}(p))$
 Compute texture resonance: $\mathbf{G} = \text{GramMatrix}(\mathbf{h})$
 Validate parameter space: $\theta_{\text{valid}} = \text{ValidateConstraints}(p)$
 Construct embedding: $\mathbf{e}_{\text{audio}} = \text{Concat}(\mathbf{h}, \mathbf{G}, p)$
 Store with metadata: $\mathcal{K}_{\text{audio}}[p] = (\mathbf{e}_{\text{audio}}, \theta_{\text{valid}}, \text{Meta}(p))$
end for
return $\mathcal{K}_{\text{audio}}$

For a toy example, two tones can achieve a high cosine similarity between mean-pooled embeddings (e.g., $\cos(\bar{\mathbf{h}}_1, \bar{\mathbf{h}}_2) \approx 0.95$) while remaining perceptually distinct, because the temporal structure of co-activations is discarded.

A Concrete Contrast. Consider two guitar tones:

1. **Sustained distortion:** a heavily overdriven tone with near-constant harmonic content.
2. **Modulated chorus:** a clean tone with slow LFO modulation, producing periodic spectral variation.

Both can exhibit similar mean-pooled embeddings (e.g., comparable average spectral centroid), yet they are perceptually and functionally distinct. Mean pooling struggles because it discards the *temporal structure* of feature co-activations that differentiates non-linear saturation from time-varying modulation.

4.2.3 THE TEXTURE HYPOTHESIS

We hypothesize that audio *timbre* is better characterized by *feature relationships* than by individual feature magnitudes. This is analogous to visual style transfer, where style is captured by feature correlations rather than pixel-level content (Gatys et al., 2015). In our setting, correlations between deep acoustic channels encode how spectral and temporal patterns co-occur (e.g., “brightness” with “roughness”), which directly relates to which classes of effects are active (distortion, modulation, dynamics, and filtering). This framing is also consistent with findings in auditory neuroscience: timbre perception depends on distributed population responses and interactions, not solely on marginal feature magnitudes.

4.2.4 TEXTURE RESONANCE RETRIEVAL (TRR)

We introduce TRR as a novel second-order statistical representation for audio texture that addresses fundamental limitations of mean-pooled embeddings in distinguishing non-stationary textures. Given an audio reference a_{ref} , we extract intermediate representations $\mathbf{H} \in \mathbb{R}^{C \times T}$ from the 9th layer of Wav2Vec2 (768-dim, pre-trained on LibriSpeech). The audio is preprocessed at 16kHz sample rate with a 25ms window and 10ms hop length.

The texture resonance is computed as the Gram matrix:

$$\mathbf{G} = \frac{1}{T} \mathbf{H} \mathbf{H}^\top \in \mathbb{R}^{C \times C} \quad (6)$$

where $\mathbf{H} \in \mathbb{R}^{C \times T}$ are the frame-level feature activations with C channels and T time steps, and $\mathbf{G}_{ij}$ captures the correlation between channels i and j across time, encoding how different acoustic features co-activate.

To ensure scale invariance and numerical stability, we apply spectral normalization:

$$\mathbf{G}_{\text{norm}} = \frac{\mathbf{G}}{\|\mathbf{G}\|_F} \quad (7)$$

Algorithm 3 Complete Texture Resonance Retrieval (TRR) Algorithm**Require:** Audio sample a , sample rate $sr = 16000$, Wav2Vec2 model $\mathcal{W}$ **Ensure:** TRR embedding $\mathbf{G}_{\text{norm}} \in \mathbb{R}^{C \times C}$ where $C = 768$ **// Step 1: Audio Preprocessing**Resample a to 16kHz if neededNormalize amplitude: $a \leftarrow a / \max(|a|)$ Ensure minimum length: $T_{\text{audio}} \geq 1$ second**// Step 2: Feature Extraction**

Extract frame-level features from layer 9 of Wav2Vec2:

 $\mathbf{H} \leftarrow \mathcal{W}_{\text{layer 9}}(a) \in \mathbb{R}^{C \times T}$ where $C = 768$ is feature dimension, T is number of time frames**// Step 3: Gram Matrix Computation**

Compute second-order statistics:

 $\mathbf{G} \leftarrow \frac{1}{T} \mathbf{H} \mathbf{H}^T \in \mathbb{R}^{C \times C}$ $\mathbf{G}_{ij}$ captures correlation between channels i and j **// Step 4: Spectral Normalization**Compute Frobenius norm: $\sigma \leftarrow \|\mathbf{G}\|_F = \sqrt{\sum_{i,j} \mathbf{G}_{ij}^2}$ Normalize for scale invariance: $\mathbf{G}_{\text{norm}} \leftarrow \mathbf{G} / \sigma$ **return** $\mathbf{G}_{\text{norm}}$ **Complexity Analysis:***Time:* $\mathcal{O}(T \cdot C)$ for feature extraction, $\mathcal{O}(C^2 \cdot T)$ for Gram matrix*Space:* $\mathcal{O}(C \cdot T)$ for features, $\mathcal{O}(C^2)$ for Gram matrix*Total:* $\mathcal{O}(C^2 \cdot T)$ time, $\mathcal{O}(C^2)$ space when features are not cached

where $\|\mathbf{G}\|_F$ is the Frobenius norm, normalizing for differences in overall signal energy.

The similarity between query and database textures is computed via cosine similarity on the normalized Gram matrices:

$$\text{sim}_{\text{TRR}}(\mathbf{G}_q, \mathbf{G}_d) = \frac{\text{tr}(\mathbf{G}_q \mathbf{G}_d^T)}{\|\mathbf{G}_q\|_F \|\mathbf{G}_d\|_F} \quad (8)$$

Invariances. By aggregating across time, TRR is approximately time-shift invariant: it preserves *what relationships occur*, not *when* they occur. The normalization step provides scale invariance to global energy.

Connection to Parameter Space. Many effect families manifest primarily as changes in channel co-activation structure rather than in the mean feature magnitude (e.g., saturation introduces correlated harmonics; chorus creates periodic correlations). TRR therefore acts as a *texture prior* for retrieving parameter presets that are more likely to activate the appropriate effect subset.

Intuition. While mean-pooled embeddings capture first-order statistics (what textures are present), Gram matrices capture second-order statistics (how textures relate to each other), which is crucial for distinguishing effects like distortion (harmonic relationships) and modulation (temporal correlations).

[Figure: TRR Feature Extraction Pipeline. This figure should illustrate the complete process: (1) Input audio waveform at 16kHz, (2) Wav2Vec2 feature extraction from layer 9 producing $\mathbf{H} \in \mathbb{R}^{768 \times T}$, (3) Gram matrix computation $\mathbf{G} = \frac{1}{T} \mathbf{H} \mathbf{H}^T$, (4) Spectral normalization to produce $\mathbf{G}_{\text{norm}}$. Show data dimensions at each stage and use arrows to indicate flow. Include a small heatmap visualization of an example Gram matrix to illustrate the second-order structure.]

Figure 5: Texture Resonance feature extraction pipeline. The process transforms audio input through Wav2Vec2 feature extraction, Gram matrix computation to capture second-order statistics, and spectral normalization for scale invariance.

[Figure: Gram Matrix Computation Visualization. This figure should show: (1) Frame-level feature matrix $\mathbf{H}$ as a 2D heatmap ($C \times T$), (2) The matrix multiplication $\mathbf{H}\mathbf{H}^\top$ with arrows showing the operation, (3) The resulting Gram matrix $\mathbf{G}$ as a symmetric $C \times C$ heatmap. Use color coding to show how correlations between different feature channels are captured. Include annotations explaining that diagonal elements capture channel variances while off-diagonal elements capture co-activation patterns.]

Figure 6: Gram matrix computation captures second-order statistics. The matrix $\mathbf{G} = \frac{1}{T}\mathbf{H}\mathbf{H}^\top$ encodes channel-to-channel correlations, where $\mathbf{G}_{ij}$ measures how features i and j co-activate across time.

4.2.5 THEORETICAL ANALYSIS OF TEXTURE RESONANCE RETRIEVAL

We provide theoretical justification for the effectiveness of TRR by analyzing its invariance properties and connection to parameter space.

Time-Invariance Property. Since TRR aggregates feature correlations across time through the Gram matrix $\mathbf{G} = \frac{1}{T}\mathbf{H}\mathbf{H}^\top$, the representation is approximately invariant to temporal shifts.

Proposition 1 (Approximate Time-Shift Invariance). For any feature matrix $\mathbf{H} \in \mathbb{R}^{C \times T}$ and its circularly time-shifted version $\mathbf{H}'$ (shifted by τ time steps), when $T \gg \tau$, we have:

$$\|\mathbf{G} - \mathbf{G}'\|_F \approx 0 \quad (9)$$

where $\mathbf{G} = \frac{1}{T}\mathbf{H}\mathbf{H}^\top$ and $\mathbf{G}' = \frac{1}{T}\mathbf{H}'(\mathbf{H}')^\top$.

Intuition: The Gram matrix aggregates correlations across all time pairs, making it insensitive to when patterns occur, only to what patterns co-occur. This is crucial for audio texture matching, where the same effect (e.g., chorus modulation) can start at different times but should be recognized as similar.

Scale-Invariance Property. The spectral normalization step ensures invariance to global energy scaling.

Proposition 2 (Scale Invariance). For any scaling factor $\alpha > 0$ and feature matrix $\mathbf{H}$, the normalized TRR embedding satisfies:

$$\text{TRR}(\alpha\mathbf{H}) = \text{TRR}(\mathbf{H}) \quad (10)$$

Proof sketch: For $\mathbf{H}' = \alpha\mathbf{H}$, we have $\mathbf{G}' = \frac{1}{T}(\alpha\mathbf{H})(\alpha\mathbf{H})^\top = \alpha^2\mathbf{G}$. Since $\|\mathbf{G}'\|_F = \alpha^2\|\mathbf{G}\|_F$, the normalized matrix is $\mathbf{G}'_{\text{norm}} = \frac{\alpha^2\mathbf{G}}{\alpha^2\|\mathbf{G}\|_F} = \frac{\mathbf{G}}{\|\mathbf{G}\|_F} = \mathbf{G}_{\text{norm}}$.

This property is essential for robustness to recording level variations and dynamic range differences between audio samples.

Connection to Parameter Space. Many audio effect families manifest primarily through changes in channel co-activation structure rather than mean feature magnitudes:

- **Saturation/Distortion:** Introduces correlated harmonics across frequency channels, creating strong off-diagonal correlations in $\mathbf{G}$
- **Chorus/Modulation:** Creates periodic temporal correlations that appear as structured patterns in the Gram matrix
- **Compression/Dynamics:** Alters variance relationships between channels, affecting diagonal and near-diagonal elements

Therefore, TRR acts as a *texture prior* for retrieving parameter presets that activate appropriate effect subsets, rather than merely matching average spectral content.

4.2.6 HYBRID RETRIEVAL ALGORITHM

For a query $q = (T_{\text{desc}}, A_{\text{ref}})$, we retrieve contextual examples from both knowledge bases:

[Figure: TRR vs. Mean Pooling Comparison. This figure should show a side-by-side comparison for two audio samples: (1) A sustained distortion tone, (2) A modulated chorus tone. For each sample, show: (a) Waveform, (b) Mean-pooled embedding as a bar chart showing average channel activations, (c) TRR Gram matrix as a heatmap. Highlight how the two samples have similar mean-pooled embeddings (similar bar heights) but distinctly different Gram matrices (different correlation patterns). Use annotations to point out key differences in co-activation structure.]

Figure 7: Comparison of mean pooling vs. Texture Resonance Retrieval. While mean pooling captures first-order statistics (average channel activations), TRR captures second-order statistics (channel co-activation patterns), enabling discrimination of textures with similar average content but different structural properties.

Algorithm 4 Dual-Modal Hybrid Retrieval with Uncertainty-Aware Fusion

Require: Query $q = (T_{\text{desc}}, A_{\text{ref}})$, knowledge bases $\mathcal{K}_{\text{text}}, \mathcal{K}_{\text{audio}}$, top-K value K

Ensure: Fused top-K retrieval results $\mathcal{R}$

// Step 1: Text Retrieval

Encode text query: $\mathbf{e}_{\text{text}} = \text{SBERT}(T_{\text{desc}})$

Retrieve top-K candidates: $\mathcal{R}_{\text{text}} = \text{TopK}(\mathcal{K}_{\text{text}}, \mathbf{e}_{\text{text}})$

Compute text uncertainty: $u_{\text{text}} = \text{Entropy}(\text{Similarities}(\mathcal{R}_{\text{text}}))$

// Step 2: Audio Retrieval (if reference available)

if A_{ref} is provided **then**

Extract audio features: $\mathbf{h} = \text{Wav2Vec2}(A_{\text{ref}})$

Compute TRR encoding: $\mathbf{G} = \text{TRR}(\mathbf{h})$

Retrieve top-K candidates: $\mathcal{R}_{\text{audio}} = \text{TopK}(\mathcal{K}_{\text{audio}}, \mathbf{G})$

Compute audio uncertainty: $u_{\text{audio}} = \text{Entropy}(\text{Similarities}(\mathcal{R}_{\text{audio}}))$

else

$\mathcal{R}_{\text{audio}} = \emptyset, u_{\text{audio}} = \infty$

end if

// Step 3: Uncertainty-Aware Weight Computation

Compute modality weights:

$w_{\text{text}} = \exp(-\beta \cdot u_{\text{text}})$

$w_{\text{audio}} = \exp(-\beta \cdot u_{\text{audio}})$ where β is temperature parameter

Normalize: $w_{\text{text}} \leftarrow \frac{w_{\text{text}}}{w_{\text{text}} + w_{\text{audio}}}$

Normalize: $w_{\text{audio}} \leftarrow \frac{w_{\text{audio}}}{w_{\text{text}} + w_{\text{audio}}}$

// Step 4: Fusion via Weighted Score Combination

Initialize fused candidates: $\mathcal{C} = \mathcal{R}_{\text{text}} \cup \mathcal{R}_{\text{audio}}$

for all candidate $c \in \mathcal{C}$ **do**

$s_{\text{text}}(c) = \text{Similarity}(c, T_{\text{desc}})$ if $c \in \mathcal{R}_{\text{text}}$ else 0

$s_{\text{audio}}(c) = \text{Similarity}(c, A_{\text{ref}})$ if $c \in \mathcal{R}_{\text{audio}}$ else 0

$s_{\text{fused}}(c) = w_{\text{text}} \cdot s_{\text{text}}(c) + w_{\text{audio}} \cdot s_{\text{audio}}(c)$

end for

// Step 5: Rank and Return Top-K

Sort $\mathcal{C}$ by s_{fused} in descending order

$\mathcal{R} \leftarrow$ Top-K candidates from sorted $\mathcal{C}$

return $\mathcal{R}$

4.2.7 THEORETICAL ANALYSIS OF DUAL-MODAL FUSION

We provide theoretical justification for our uncertainty-aware fusion strategy.

Uncertainty Estimation via Entropy. For a retrieval modality $m \in \{\text{text}, \text{audio}\}$, let $S_m = \{s_1, s_2, \dots, s_K\}$ be the similarity scores of the top-K retrieved candidates, normalized to sum to 1. We estimate retrieval uncertainty as:

$$u_m = - \sum_{i=1}^K s_i \log s_i \quad (11)$$

[Figure: Dual-Modal Retrieval Flow. This figure should show the complete dual-modal retrieval pipeline: (1) Left branch: Text description $\rightarrow$ SBERT encoding $\rightarrow$ Text retrieval from $\mathcal{K}_{\text{text}} \rightarrow$ Top-K text results with uncertainty u_{text} ; (2) Right branch: Audio reference $\rightarrow$ Wav2Vec2 + TRR encoding $\rightarrow$ Audio retrieval from $\mathcal{K}_{\text{audio}} \rightarrow$ Top-K audio results with uncertainty u_{audio} ; (3) Center: Uncertainty-aware weight computation ($w_{\text{text}}, w_{\text{audio}}$); (4) Bottom: Fusion module combining weighted scores to produce final Top-K results. Use color coding for text (blue) and audio (orange) pathways, with arrows showing data flow.]

Figure 8: Dual-modal retrieval pipeline with uncertainty-aware fusion. The system retrieves candidates from both text and audio knowledge bases, estimates retrieval uncertainty for each modality, and fuses results using dynamic weights that down-weight unreliable modalities.

[Figure: Modality Fusion Mechanism. This figure should illustrate the fusion process: (1) Top: Two similarity distributions for text and audio retrieval results, visualized as histograms or bar charts; (2) Middle: Uncertainty computation showing how entropy is calculated from similarity distributions, with annotations showing high uncertainty (flat distribution) vs. low uncertainty (peaked distribution); (3) Bottom: Weight computation using exponential decay $w = \exp(-\beta \cdot u)$ with a curve showing how uncertainty maps to weight. Include a concrete example showing how vague text (high u_{text} , low w_{text}) gets down-weighted while confident audio (low u_{audio} , high w_{audio}) dominates the fusion.]

Figure 9: Uncertainty-aware modality fusion mechanism. Retrieval uncertainty is estimated from similarity distributions using entropy, then mapped to fusion weights via exponential decay. This allows the system to automatically down-weight unreliable modalities (e.g., vague text prompts) and rely more on confident modalities.

Intuition: High entropy indicates a flat similarity distribution, suggesting the retrieval is uncertain (many equally plausible candidates). Low entropy indicates a peaked distribution, suggesting high confidence (one clear best match).

Adaptive Weight Computation. We map uncertainty to fusion weights using exponential decay:

$$w_m = \frac{\exp(-\beta \cdot u_m)}{\sum_{m'} \exp(-\beta \cdot u_{m'})} \quad (12)$$

where $\beta > 0$ controls the sensitivity to uncertainty.

Proposition 3 (Monotonicity). The fusion weight w_m is monotonically decreasing in uncertainty u_m :

$$\frac{\partial w_m}{\partial u_m} < 0 \quad (13)$$

Proof sketch: Since $\exp(-\beta \cdot u_m)$ is monotonically decreasing in u_m , and the softmax normalization preserves monotonicity for two modalities, the result follows.

Robustness to Missing Modalities. When one modality is absent (e.g., $A_{\text{ref}} = \emptyset$), we set $u_{\text{audio}} = \infty$, which yields $w_{\text{audio}} = 0$ and $w_{\text{text}} = 1$, gracefully falling back to text-only retrieval.

Robustness to Degraded Inputs. When one modality provides low-quality input (e.g., vague text like “make it sound good”), the retrieval produces a flat similarity distribution with high entropy $u_{\text{text}} \gg u_{\text{audio}}$. The fusion automatically down-weights the unreliable modality:

$$w_{\text{text}} \approx 0, \quad w_{\text{audio}} \approx 1 \quad (14)$$

This explains the robustness results in Table 8, where Audio-Agent remains stable under vague prompts by relying on audio-grounded retrieval.

4.3 ADAPTIVE PREFERENCE LEARNING

We view preference adaptation as a natural extension of the retrieval loop, but it is not exercised in the current experimental evaluation. The focus of this paper is on retrieval and reasoning under fixed knowledge bases, while personalized feedback-driven updates are left as future work.

4.4 PARAMETER SPACE CONSTRAINTS AND GENERATION

A critical challenge in LLM-based parameter generation is ensuring that outputs remain within physically valid and musically meaningful ranges. Unconstrained generation can produce parameters that violate plugin specifications (e.g., negative frequencies, ratios below 1:1) or are technically valid but perceptually incoherent (e.g., extreme settings that produce silence or clipping).

4.4.1 CONSTRAINT FORMALIZATION

We formalize the parameter space as $\Theta = \prod_{i=1}^d [\theta_i^{\min}, \theta_i^{\max}]$, where each parameter θ_i has a valid range $[\theta_i^{\min}, \theta_i^{\max}]$ defined by the plugin specification. Additionally, we enforce cross-parameter constraints $\mathcal{C}$ that capture dependencies between parameters (e.g., attack time must be less than release time in envelope-based effects).

Types of Constraints.

1. **Range Constraints:** $\theta_i \in [\theta_i^{\min}, \theta_i^{\max}]$ for all $i \in \{1, \dots, d\}$
2. **Dependency Constraints:** $\theta_i \leq \theta_j$ for specific parameter pairs (e.g., attack $\leq$ release)
3. **Activation Constraints:** If effect E_k is inactive, all its parameters must be at default values
4. **Semantic Constraints:** Parameter combinations must be musically meaningful (e.g., compressor with ratio=1:1 and threshold=0dB is effectively bypassed)

4.4.2 PARAMETER GENERATION WITH CONSTRAINT SATISFACTION

We employ a retrieval-guided generation strategy that combines LLM reasoning with explicit constraint checking and repair.

[Figure: Constraint Satisfaction Mechanism. This figure should illustrate the constraint checking and repair process: (1) Top: LLM generates raw parameters shown as a parameter vector with some values highlighted in red (violations); (2) Middle: Constraint validation showing range checks (min/max bounds) and dependency checks (e.g., attack $\leq$ release) with arrows pointing to violations; (3) Bottom: Repair process showing clipping for range violations and interpolation with retrieved references for dependency violations. Use color coding: green for valid parameters, red for violations, yellow for repaired parameters. Include a flowchart showing the decision logic: if violations exist $\rightarrow$ repair $\rightarrow$ re-validate $\rightarrow$ fallback if needed.]

Figure 10: Constraint satisfaction mechanism. The system validates generated parameters against range and dependency constraints, repairs violations through clipping and reference-guided interpolation, and falls back to retrieved references if repair fails.

[Figure: Parameter Space Mapping. This figure should visualize the mapping from retrieval results to parameter space: (1) Left: Retrieved presets shown as points in a 2D projection of parameter space (use t-SNE or PCA), color-coded by style/genre; (2) Center: Query point (text + audio) shown as a star, with arrows pointing to top-K retrieved neighbors; (3) Right: Generated parameter shown as a new point, with a convex hull or shaded region indicating the valid parameter space Θ . Show constraint boundaries as dashed lines. Include annotations explaining how retrieval constrains the generation to a feasible neighborhood, and how constraint repair projects invalid outputs back into Θ .]

Figure 11: Parameter space mapping from retrieval to generation. Retrieved presets define a neighborhood in parameter space that guides LLM generation. Constraint satisfaction ensures the output remains within the valid parameter space Θ while preserving proximity to retrieved references.

4.4.3 THEORETICAL GUARANTEES

Constraint Satisfaction Guarantee. Theorem 1 (Constraint Satisfaction). Algorithm 6 always produces a parameter vector $\theta_{\text{repaired}} \in \Theta$ that satisfies all constraints in $\mathcal{C}$.

Algorithm 5 Constrained Parameter Generation**Require:** Query $q = (T_{\text{desc}}, A_{\text{ref}})$, retrieved context $\mathcal{R}$, constraint set $\mathcal{C}$ **Ensure:** Valid parameter vector $\theta^* \in \Theta$ **// Step 1: Build Contextual Prompt**Extract reference parameters: $\{\theta_1, \theta_2, \dots, \theta_K\} \leftarrow \mathcal{R}$ Format prompt: $P \leftarrow \text{FormatPrompt}(T_{\text{desc}}, \{\theta_1, \dots, \theta_K\}, \mathcal{C})$ *Include constraint specifications and valid ranges in prompt***// Step 2: LLM Generation with Structured Output**Generate candidate: $\theta_{\text{raw}} \leftarrow \text{LLM}(P)$ Parse JSON output: $\theta_{\text{parsed}} \leftarrow \text{ParseJSON}(\theta_{\text{raw}})$ **// Step 3: Constraint Validation**violations $\leftarrow \emptyset$ **for all** parameter θ_i in θ_{parsed} **do** **if** $\theta_i < \theta_i^{\min}$ OR $\theta_i > \theta_i^{\max}$ **then** Add $(i, \text{"range"})$ to violations **end if****end for****for all** constraint $c \in \mathcal{C}$ **do** **if** $\neg c(\theta_{\text{parsed}})$ **then** Add $(c, \text{"dependency"})$ to violations **end if****end for****// Step 4: Constraint Repair****if** violations $\neq \emptyset$ **then** $\theta^* \leftarrow \text{RepairConstraints}(\theta_{\text{parsed}}, \text{violations}, \mathcal{R})$ **else** $\theta^* \leftarrow \theta_{\text{parsed}}$ **end if****return** θ^* **Complexity Analysis:***Time:* $\mathcal{O}(d)$ for range checks, $\mathcal{O}(|\mathcal{C}|)$ for dependency checks*Space:* $\mathcal{O}(d)$ for parameter storage*LLM inference cost dominates, but is amortized across queries*

Proof sketch: Range clipping ensures $\theta_i \in [\theta_i^{\min}, \theta_i^{\max}]$ for all i . For dependency constraints, if interpolation fails, the fallback to the nearest retrieved reference guarantees constraint satisfaction since all references in $\mathcal{R}$ are pre-validated. $\square$

Proximity Preservation. While constraint repair modifies the generated parameters, it aims to preserve proximity to the original LLM output. We quantify this via the repair distance:

$$\Delta_{\text{repair}} = \|\theta_{\text{repaired}} - \theta_{\text{invalid}}\|_2 \quad (15)$$

In our experiments (Table 1), the average repair distance is small (typically $\leq 5\%$ of parameter range), indicating that most LLM outputs are nearly valid and require only minor corrections.

4.5 COMPLETE INFERENCE PIPELINE

The full parameter inference process combines retrieval, reasoning, and constraint satisfaction:

Complexity Analysis: The inference cost scales with knowledge base size, embedding dimensionality, and generation length. In our experimental setting, retrieval and generation are lightweight, and we focus on comparative performance rather than tight runtime guarantees.

Algorithm 6 Constraint Repair via Nearest Valid Projection

Require: Invalid parameter θ_{invalid} , violations, retrieved references $\mathcal{R}$
Ensure: Repaired parameter $\theta_{\text{repaired}} \in \Theta$
Initialize: $\theta_{\text{repaired}} \leftarrow \theta_{\text{invalid}}$
// Step 1: Fix Range Violations via Clipping
for all $(i, \text{"range"}) \in \text{violations}$ **do**
 $\theta_{\text{repaired}}[i] \leftarrow \text{clip}(\theta_{\text{invalid}}[i], \theta_i^{\min}, \theta_i^{\max})$
end for
// Step 2: Fix Dependency Violations
for all $(c, \text{"dependency"}) \in \text{violations}$ **do**
 Find nearest reference satisfying c : $\theta_{\text{ref}} \leftarrow \arg \min_{\theta \in \mathcal{R}, c(\theta)} \|\theta - \theta_{\text{repaired}}\|_2$
 Interpolate: $\theta_{\text{repaired}} \leftarrow 0.7 \cdot \theta_{\text{repaired}} + 0.3 \cdot \theta_{\text{ref}}$
 Weighted blend favors generated values while ensuring constraint satisfaction
end for
// Step 3: Verify All Constraints
if $\exists c \in \mathcal{C} : \neg c(\theta_{\text{repaired}})$ **then**
 Fallback: Use nearest retrieved reference
 $\theta_{\text{repaired}} \leftarrow \arg \min_{\theta \in \mathcal{R}} \|\theta - \theta_{\text{invalid}}\|_2$
end if
return θ_{repaired}

Algorithm 7 Audio-Agent Parameter Inference

Require: Query $q = (T_{\text{desc}}, A_{\text{ref}})$, history $\mathcal{H}$, knowledge base $\mathcal{K}$
Ensure: Parameter vector θ^*
Preprocess query: $q_{\text{proc}} = \text{Preprocess}(q)$
Retrieve context: $\mathcal{R} = \text{DualModalRetrieval}(q_{\text{proc}}, \mathcal{K})$
Build context: $\text{Context} = \text{FormatContext}(\mathcal{R}, \mathcal{H})$
Generate parameters: $\theta^* = \text{LLM}(q_{\text{proc}}, \text{Context})$
Validate output: $\theta^* \leftarrow \text{Validate}(\theta^*)$
return θ^*

5 EXPERIMENTS

5.1 EXPERIMENTAL SETUP

We construct an evaluation set of 50 guitar tone samples by merging the parameter database with the corresponding audio-vector database. Each item includes a style description, parameter preset, and audio vector used for retrieval and evaluation.

[Figure: Experimental Setup Pipeline. This figure should illustrate the complete experimental setup: (1) Top-left: Raw parameter database showing guitar preset parameters (gain, EQ, compression, etc.); (2) Top-right: Audio database showing corresponding audio vectors and TRR Gram matrices; (3) Bottom-left: Dataset construction showing merging process and split into training/test sets; (4) Bottom-right: Evaluation pipeline showing query processing, retrieval, parameter generation, and metric computation. Use arrows to show data flow and color coding for different data types.]

Figure 12: Experimental setup pipeline. The figure shows the construction of the evaluation dataset from parameter and audio databases, and the evaluation pipeline used for comparative analysis.

Baselines: (1) *Zero-shot*: direct parameter generation without retrieval. (2) *Text-RAG*: retrieval using text descriptions only. (3) *Vector-RAG*: retrieval using audio vectors only. (4) *Audio-Agent*: our texture-aware dual retrieval with candidate fusion.

Table 4: Experimental Hyperparameters

Parameter	Value	Description
Sample rate	16 kHz	Audio preprocessing sample rate
Window size	25 ms	STFT window size
Hop size	10 ms	STFT hop size
Wav2Vec2 layer	9	Feature extraction layer
Feature dimension	768	Wav2Vec2 feature dimension
Retrieval top-K	5	Number of retrieved candidates
Fusion weight β	1.0	Uncertainty temperature parameter
Text embedding	SBERT-base	Text encoder model

Table 5: Main Results on 50 Guitar Tone Samples

Method	Param. Dist. (L2)	Acc@0.1	Cos	Active Recall
Zero-shot	42.67	0.12	0.45	0.15
Text-RAG	41.37	0.15	0.48	0.18
Vector-RAG	22.27	0.38	0.72	0.41
Audio-Agent (Ours)	21.86	0.45	0.78	0.49

5.2 RQ1: OVERALL EFFECTIVENESS

We evaluate Audio-Agent against three baseline methods across comprehensive objective metrics. Table 5 presents the primary evaluation results on our 50-style guitar dataset.

What. Table 5 shows that Audio-Agent substantially reduces parameter error compared to the zero-shot baseline and improves tolerance accuracy, direction consistency, and active-parameter recall.

Why. Retrieval reduces the difficulty from inventing parameters to adapting known-good exemplars. The strong gap between Text-RAG and Vector-RAG indicates that audio evidence provides tighter physical constraints than ambiguous text.

So What. The large gain in *Active Recall* suggests that the system is better at identifying which effects should be engaged (a critical property for sparse parameter vectors).

5.2.1 WHY DOES TEXT-ONLY PERFORM SO POORLY?

Text-RAG’s marginal improvement over zero-shot reflects semantic ambiguity in production descriptors. For example, a query like “*warm vintage breakup*” can retrieve stylistically plausible but functionally mismatched references such as “Warm Jazz Clean” (warm but insufficient gain), “Vintage Rock” (vague and under-specified), or “British Crunch” (correct family but wrong breakup point). Without audio grounding, text retrieval cannot disambiguate which interpretation matches the intended texture.

5.2.2 WHERE DOES TRR HELP?

TRR provides the most benefit when the target sound is defined by interaction patterns (e.g., breakup dynamics, modulation, or subtle presence shaping) rather than by average spectral content. In these cases, mean-pooled embeddings can retrieve broadly similar tones while missing discriminative temporal co-activation structure. We analyze such texture-sensitive long-tail behavior in RQ2 via the “Stadium Rock” case study (Table 6).

5.3 METRICS

We evaluate on four metrics aligned with our implementation: (1) *Parameter Distance* (RMSE over flattened parameter vectors), (2) *Acc@0.1* (fraction of parameters within a tolerance of 0.1), (3) *Cosine Similarity* (direction consistency between parameter vectors), and (4) *Active Recall* (recall over non-zero parameters).

[Figure: Metric Computation Visualization. This figure should show the computation process for each metric: (1) Top-left: Parameter Distance - show two parameter vectors θ_{pred} and θ_{gt} , with L2 distance formula and geometric interpretation; (2) Top-right: Acc@0.1 - show parameter-wise absolute differences, with highlighting of parameters within tolerance (< 0.1) vs. outside tolerance; (3) Bottom-left: Cosine Similarity - show parameter vectors in high-dimensional space, with angle computation formula; (4) Bottom-right: Active Recall - show confusion matrix comparing active (non-zero) parameters between predicted and ground truth, with recall computation. Use color coding: green for correct, red for incorrect.]

Figure 13: Evaluation metrics computation. The figure illustrates how each metric is computed from predicted and ground-truth parameter vectors.

5.4 RQ2: TEXTURE VALIDITY AND MECHANISM

To validate the effectiveness of our proposed Texture Resonance Retrieval (TRR) against traditional Vector Similarity, we conduct comparative experiments using synthetic audio data. Table 6 presents the retrieval error (L2 distance) for two distinct musical styles.

Table 6: Case Study: Texture Resonance vs. Vector Baseline (Lower is Better)

Style	Test Sample	Vector-RAG	Texture-RAG (Ours)
Clean	Stadium Rock	18.51	8.01
Distortion	Punk Rock Raw	53.31	51.14
Vintage	Tweed Breakup	10.72	7.74

What. Table 6 highlights texture-sensitive cases where TRR improves parameter alignment, especially for subtle overdrive dynamics and clean textures where mean pooling can over-smooth discriminative cues.

Why. Mean pooling emphasizes average content and can conflate signals that share similar global statistics. TRR instead captures channel co-activation structure that is sensitive to non-linear harmonic coupling and modulation.

So What. These results support the Texture Hypothesis: correlation-based descriptors provide a better inductive bias for texture-grounded retrieval in long-tail timbre design.

Interpretation. TRR is not a universal substitute for vector similarity; instead, it complements it by favoring exemplars that preserve co-activation structure. This is particularly useful in neighborhoods where multiple presets share similar mean content but differ in second-order interaction patterns.

5.4.1 CASE STUDY: “STADIUM ROCK” RETRIEVAL GRANULARITY

We analyze the “Stadium Rock” sample (Table 6) to illustrate a typical retrieval granularity failure mode for vector embeddings.

Ground truth. “Stadium Rock” is characterized by a bright, clean tone with pronounced high-mid presence that needs to be expressed via subtle EQ and dynamics choices, rather than by changing the overall effect family.

Retrieval. In Vector-RAG, the nearest neighbor can be dominated by broad spectral similarity (e.g., “clean guitar”), retrieving an example such as “Funk Clean” that matches overall brightness but misses the specific “stadium” presence profile. In contrast, TRR retrieval is more sensitive to co-activation patterns in upper-mid frequency channels, yielding a closer reference neighborhood.

Prediction. When the retrieved context is too coarse, the generator tends to copy a plausible clean preset but fails to recover the small, perceptually salient presence shaping needed for the target. With

Table 7: Feature–Parameter Alignment (Spearman ρ ; higher is better)

Feature Space	All Pairs	Non-Stationary Pairs
Mean Vector (Vector-RAG)	0.205	0.341
Texture Resonance (TRR)	-0.005	0.036

Table 8: Robustness Under Vague Text

Condition	Text-RAG	Audio-Agent (Ours)
Vague Text	45.35	26.17

TRR-grounded context, the inferred parameters better preserve the interaction pattern associated with the target texture, reducing the downstream parameter error in this case.

5.4.2 FEATURE–PARAMETER ALIGNMENT

To quantify how well feature similarity predicts parameter similarity, we compute pairwise correlations between (i) feature-space similarity and (ii) cosine similarity between ground-truth parameter vectors across all sample pairs. We report Spearman’s ρ for all pairs and a subset focusing on distortion/modulation-heavy (non-stationary) tones.

We observe that global feature–parameter alignment is weak for TRR, consistent with the many-to-one nature of timbre: multiple parameter configurations can yield perceptually similar textures. This motivates our framing of TRR as a *retrieval prior* that helps on texture-sensitive long-tail cases (Table 6) rather than as a globally calibrated proxy for parameter similarity.

Across qualitative inspection, we observe three recurring failure modes. First, tones involving effect types not well covered by the preset database can lead to retrieval mismatch and over-regularized generation, as the system defaults to the nearest available exemplar rather than extrapolating to unseen combinations. Second, text descriptors can bias retrieval toward a genre-consistent but texture-inconsistent neighborhood; for instance, queries containing “vintage” may pull clean references when breakup is actually required, conflating stylistic era with technical effect type. Third, small but perceptually salient deviations, such as a presence boost around 2–3 kHz, can be hard to recover without higher-resolution conditioning or explicit EQ-band priors, as these subtle adjustments may not manifest strongly in global texture embeddings.

5.5 ABLATION STUDY

To isolate the contribution of each component, we compare Text-RAG, Vector-RAG, and Texture-RAG configurations under the same evaluation metrics.

Analysis: We isolate component contributions by comparing Text-RAG, Vector-RAG, and Texture-RAG (Ours). The improvements from Vector-RAG to Texture-RAG indicate the benefit of texture-aware retrieval, while the gap between Text-RAG and Vector-RAG highlights the importance of audio evidence in parameter inference.

5.6 PERCEPTUAL LISTENING TEST (PLANNED)

We have prepared a MUSHRA-style evaluation interface for subjective assessment, but a full listening study and statistical analysis are left to future work. In the current submission, we focus on objective parameter-based evaluation.

5.7 RQ3: ROBUSTNESS UNDER DEGRADED INPUTS

We evaluate robustness under a text-degradation scenario that mirrors ambiguous user prompts. We replace the prompt with a vague instruction (e.g., “Make it sound good with some effects.”) and compare Text-RAG with our texture-aware retrieval.

What. Under vague prompts, text-only retrieval collapses, while Audio-Agent remains substantially more stable.

Why. The fusion policy can down-weight low-information text and rely on audio-grounded retrieval.

So What. This behavior is essential for real-world usage, where user prompts are often under-specified and noisy.

Interpretation. These results suggest that modality reliability should be treated as a first-class signal: when text collapses to generic intent, audio-conditioned retrieval can still constrain feasible parameter neighborhoods and stabilize generation.

6 DISCUSSION

6.1 BROADER IMPLICATIONS

Audio-Agent suggests a practical pattern for grounded creative assistants: treat the LLM as a reasoning layer, while using retrieval over real presets to provide feasibility constraints and stylistic priors. This reduces hallucination and keeps outputs editable within standard production workflows. More broadly, it shifts the emphasis from open-ended generation to data-efficient adaptation, improving sample efficiency (reuse of real presets), interpretability (traceable references), and controllability (parameter-level editability).

6.2 REVISITING THE TEXTURE HYPOTHESIS

We observe that texture-aware retrieval helps on specific long-tail cases (Table 6) and remains robust under vague prompts (Table 8). However, Table 7 shows that TRR similarity is not a globally calibrated proxy for parameter similarity. This motivates a careful interpretation: TRR provides a *retrieval prior* that can be beneficial in texture-sensitive neighborhoods, rather than a universal metric for parameter transfer.

An open direction is whether analogous second-order representations help bridge modality gaps in other continuous control settings (e.g., video textures, haptic signals, or physiological time series), where the “style” of a signal may be expressed through co-activation structure rather than pointwise magnitudes. Future work can validate this hypothesis via controlled ablations on non-stationarity and by pairing retrieval with human preference judgments.

Our approach has several limitations. First, retrieval quality fundamentally depends on database diversity; out-of-distribution tones that are not well-represented in the preset corpus remain challenging to handle. Second, while richer databases improve coverage, they also increase retrieval cost and require careful indexing and caching strategies to maintain acceptable query latencies. Third, the parameter identifiability problem persists: multiple parameter configurations can yield perceptually similar textures, making objective parameter distance an imperfect surrogate for perceptual match. Fourth, the subjectivity of tone quality means that what constitutes a “good” sound depends heavily on musical context and user intent, which objective metrics cannot fully capture. Fifth, the editability–synthesis trade-off is fundamental: constraining outputs to editable presets improves usability and workflow integration but may limit the reachable sound space compared to direct waveform synthesis approaches. Finally, our evaluation scope focuses on objective metrics derived from parameter alignment; controlled listening tests remain necessary to measure perceptual preference and validate whether the generated parameters align with human judgment of sonic quality.

6.3 ETHICAL CONSIDERATIONS

Audio-Agent is intended to augment user creativity rather than replace it. Risks include homogenization toward common exemplar neighborhoods and over-reliance on suggested settings. In deployment, the interface should emphasize transparency (e.g., showing retrieved references) and user agency (e.g., direct editability and opt-out of retrieval). More generally, any model or retrieval corpus built from existing recordings or presets should consider copyright, licensing, and data provenance constraints.

7 CONCLUSION

We presented Audio-Agent, a neuro-symbolic system for generating editable DSP parameters from natural language descriptions with dual-modal, texture-aware retrieval. Across objective metrics (Table 5) and robustness settings (Table 8), retrieval grounding improves parameter alignment compared to text-only or zero-shot baselines.

7.1 IMMEDIATE NEXT STEPS

In the short term, the most actionable next steps are (i) expanding preset coverage for under-represented effect families, (ii) improving transparency by exposing reference audio and parameter deltas, and (iii) validating retrieval quality with controlled listening tests.

7.2 ROADMAP

Looking forward, we envision a three-stage development path. In the **short term**, we plan to expand preset coverage for under-represented effect families and improve retrieval transparency by surfacing reference audio and parameter deltas to the user, enabling better understanding of how the system arrives at its recommendations. In the **medium term**, we aim to integrate controlled listening tests to validate perceptual quality and develop richer conditioning mechanisms that better capture time-varying structure in audio processing chains. Over the **long term**, we seek to generalize beyond guitar effects to broader production parameter spaces, including mixing, mastering, and synthesis workflows, thereby enabling grounded, editable control across diverse creative applications.

We hope this approach lowers the barrier to expressive sound design and broadens creative access to professional-grade production control.

ACKNOWLEDGMENTS

We thank the anonymous reviewers for their valuable feedback.

A ADDITIONAL ALGORITHM DETAILS

A.1 COMPLETE TRR ALGORITHM

A.2 COMPLETE DUAL-MODAL RETRIEVAL ALGORITHM

B EXPERIMENTAL PARAMETERS

Table 9 lists all hyperparameters used in our experiments.

C ADDITIONAL EXPERIMENTAL RESULTS

C.1 PER-PARAMETER ANALYSIS

Table 10 shows the performance breakdown by parameter type.

C.2 ABLATION STUDY RESULTS

Table 11 shows the results of our ablation study.

Algorithm 8 Complete Texture Resonance Retrieval Algorithm**Require:** Audio sample a , sample rate $sr = 16000$, Wav2Vec2 model $\mathcal{W}$ **Ensure:** TRR embedding $\mathbf{G}_{\text{norm}} \in \mathbb{R}^{C \times C}$ where $C = 768$ **// Step 1: Audio Preprocessing**Resample a to 16kHz if neededNormalize amplitude: $a \leftarrow a / \max(|a|)$ Ensure minimum length: $T_{\text{audio}} \geq 1$ second**// Step 2: Feature Extraction**

Extract frame-level features from layer 9 of Wav2Vec2:

 $\mathbf{H} \leftarrow \mathcal{W}_{\text{layer 9}}(a) \in \mathbb{R}^{C \times T}$ where $C = 768$ is feature dimension, T is number of time frames**// Step 3: Gram Matrix Computation**

Compute second-order statistics:

 $\mathbf{G} \leftarrow \frac{1}{T} \mathbf{H} \mathbf{H}^T \in \mathbb{R}^{C \times C}$ $\mathbf{G}_{ij}$ captures correlation between channels i and j **// Step 4: Spectral Normalization**Compute Frobenius norm: $\sigma \leftarrow \|\mathbf{G}\|_F = \sqrt{\sum_{i,j} \mathbf{G}_{ij}^2}$ Normalize for scale invariance: $\mathbf{G}_{\text{norm}} \leftarrow \mathbf{G} / \sigma$ **return** $\mathbf{G}_{\text{norm}}$ **Complexity Analysis:***Time:* $\mathcal{O}(T \cdot C)$ for feature extraction, $\mathcal{O}(C^2 \cdot T)$ for Gram matrix*Space:* $\mathcal{O}(C \cdot T)$ for features, $\mathcal{O}(C^2)$ for Gram matrix*Total:* $\mathcal{O}(C^2 \cdot T)$ time, $\mathcal{O}(C^2)$ space when features are not cached

Table 9: Complete Hyperparameters List

Parameter	Value	Description
Audio Preprocessing		
Sample rate	16 kHz	Audio preprocessing sample rate
Window size	25 ms	STFT window size
Hop size	10 ms	STFT hop size
Feature Extraction		
Wav2Vec2 model	facebook/wav2vec2-base	Pretrained model
Wav2Vec2 layer	9	Feature extraction layer
Feature dimension	768	Wav2Vec2 feature dimension
TRR dimension	768×768	Gram matrix size
Retrieval		
Text embedding	SBERT-base	Text encoder model
Top-K	5	Number of retrieved candidates
Fusion weight β	1.0	Uncertainty temperature parameter
Similarity metric	Cosine	Embedding similarity measure
Parameter Generation		
LLM model	GPT-3.5-turbo	Language model
Max tokens	1024	Maximum output tokens
Temperature	0.7	Sampling temperature
Top-p	0.9	Nucleus sampling parameter

D IMPLEMENTATION DETAILS**D.1 WEBSOCKET COMMUNICATION PROTOCOL**

The WebSocket communication between Python agent and C++ client uses a JSON-based message format.

Message Format:

Algorithm 9 Complete Dual-Modal Retrieval with Uncertainty-Aware Fusion

Require: Query $q = (T_{\text{desc}}, A_{\text{ref}})$, knowledge bases $\mathcal{K}_{\text{text}}, \mathcal{K}_{\text{audio}}$, top-K value K
Ensure: Fused top-K retrieval results $\mathcal{R}$

// Step 1: Text Retrieval
 Encode text query: $\mathbf{e}_{\text{text}} = \text{SBERT}(T_{\text{desc}})$
 Retrieve top- K candidates: $\mathcal{R}_{\text{text}} = \text{TopK}(\mathcal{K}_{\text{text}}, \mathbf{e}_{\text{text}})$
 Compute text uncertainty: $u_{\text{text}} = \text{Entropy}(\text{Similarities}(\mathcal{R}_{\text{text}}))$

// Step 2: Audio Retrieval (if reference available)
if A_{ref} is provided **then**
 Extract audio features: $\mathbf{h} = \text{Wav2Vec2}(A_{\text{ref}})$
 Compute TRR encoding: $\mathbf{G} = \text{TRR}(\mathbf{h})$
 Retrieve top- K candidates: $\mathcal{R}_{\text{audio}} = \text{TopK}(\mathcal{K}_{\text{audio}}, \mathbf{G})$
 Compute audio uncertainty: $u_{\text{audio}} = \text{Entropy}(\text{Similarities}(\mathcal{R}_{\text{audio}}))$
else
 $\mathcal{R}_{\text{audio}} = \emptyset, u_{\text{audio}} = \infty$
end if

// Step 3: Uncertainty-Aware Weight Computation
 Compute modality weights:
 $w_{\text{text}} = \exp(-\beta \cdot u_{\text{text}})$
 $w_{\text{audio}} = \exp(-\beta \cdot u_{\text{audio}})$ *where β is temperature parameter*
 Normalize: $w_{\text{text}} \leftarrow \frac{w_{\text{text}}}{w_{\text{text}} + w_{\text{audio}}}$
 Normalize: $w_{\text{audio}} \leftarrow \frac{w_{\text{audio}}}{w_{\text{text}} + w_{\text{audio}}}$

// Step 4: Fusion via Weighted Score Combination
 Initialize fused candidates: $\mathcal{C} = \mathcal{R}_{\text{text}} \cup \mathcal{R}_{\text{audio}}$
for all candidate $c \in \mathcal{C}$ **do**
 $s_{\text{text}}(c) = \text{Similarity}(c, T_{\text{desc}})$ **if** $c \in \mathcal{R}_{\text{text}}$ **else** 0
 $s_{\text{audio}}(c) = \text{Similarity}(c, A_{\text{ref}})$ **if** $c \in \mathcal{R}_{\text{audio}}$ **else** 0
 $s_{\text{fused}}(c) = w_{\text{text}} \cdot s_{\text{text}}(c) + w_{\text{audio}} \cdot s_{\text{audio}}(c)$
end for

// Step 5: Rank and Return Top-K
 Sort $\mathcal{C}$ by s_{fused} in descending order
 $\mathcal{R} \leftarrow$ Top-K candidates from sorted $\mathcal{C}$
return $\mathcal{R}$

Table 10: Per-parameter error analysis (L2 distance)

Parameter Type	Vector-RAG	Audio-Agent (Ours)
Gain/Volume	8.23	6.45
EQ Low/Mid/High	12.34	9.87
Compression Ratio	1.45	0.98
Compression Threshold	5.67	3.21
Distortion Drive	15.23	11.45
Time-based Effects	18.90	14.23

```

{
  "type": "parameter_update",
  "timestamp": 1234567890.123,
  "parameters": {
    "eq_low_gain": 2.5,
    "eq_mid_freq": 1200.0,
    "comp_threshold": -18.5,
    "comp_ratio": 4.3,
    ...
  },
  "metadata": {
    "query": "warm vintage breakup",

```

Table 11: Ablation study results (Parameter Distance)

Configuration	Error	Notes
Full Audio-Agent	21.86	All components
- Text modality	23.45	Audio-only retrieval
- Audio modality	35.67	Text-only retrieval
- TRR (use mean pooling)	24.89	Standard vector retrieval
- Uncertainty weighting	22.34	Fixed 50/50 weights
- Constraint repair	23.12	No constraint enforcement

```

    "retrieved_presets": ["Tweed Clean", "British Crunch"],
    "confidence": 0.87
  }
}

```

Acknowledgment Format:

```

{
  "type": "ack",
  "status": "success" | "error",
  "message": "Parameters applied successfully" | "Invalid range for comp_ratio",
  "applied_parameters": {...}
}

```

D.2 PERFORMANCE OPTIMIZATION

Caching Strategy. We cache TRR embeddings for all database items to enable fast retrieval. The cache is stored as a memory-mapped numpy array for efficient random access.

Parallelization. Text and audio retrieval are performed in parallel using Python’s multiprocessing module. This reduces query latency by approximately 40%.

Batch Processing. For batch inference, we process multiple queries simultaneously using vectorized operations, improving throughput by 3-4x compared to sequential processing.

REFERENCES

- Andrea Agostinelli, Timo I Denk, Zalán Borsos, Jesse Engel, Mauro Verzetti, Antoine Caillon, Qingqing Huang, Aren Jansen, Adam Roberts, Marco Tagliasacchi, et al. Musiclm: Generating music from text. *arXiv preprint arXiv:2301.11325*, 2023.
- Alexei Baevski, Henry Zhou, Abdelrahman Mohamed, and Michael Auli. wav2vec 2.0: A framework for self-supervised learning of speech representations. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pp. 12449–12460, 2020.
- Emmanouil Benetos et al. St-ito: Controlling audio effects for style transfer with inference-time optimization. *arXiv preprint arXiv:2410.21233*, 2024.
- K. Chen et al. Reclap: Improving zero shot audio classification by describing sounds. *arXiv preprint arXiv:2409.09213*, 2024.
- Michael Clemens and Ana Marasović. Mixassist: An audio-language dataset for co-creative ai assistance in music mixing. In *Conference on Language Modeling (COLM)*, 2025.
- A. Dees et al. Generative latent spaces for neural audio synthesis. 2025.
- Seunghoon Doh, Junghyun Koo, Marco A Martínez-Ramírez, Woosung Choi, Wei-Hsiang Liao, Qiyu Wu, Juhan Nam, and Yuki Mitsufuji. Can large language models predict audio effects parameters from natural language? *arXiv preprint arXiv:2505.20770*, 2025a.

- Seunghoon Doh, Junghyun Koo, Marco A Martínez-Ramírez, Woosung Choi, Wei-Hsiang Liao, Qiyu Wu, Juhan Nam, and Yuki Mitsufuji. Llm2fx-tools: Tool calling for music post-production. *arXiv preprint arXiv:2512.01559*, 2025b.
- B. Elizalde, S. Deshmukh, M. Al Ismail, and H. Wang. Clap: Learning audio concepts from natural language supervision. 2023.
- Jesse Engel, Lamtharn Hantrakul, Chenjie Gu, and Adam Roberts. Ddsp: Differentiable digital signal processing. In *International Conference on Learning Representations (ICLR)*, 2020.
- Leon A Gatys, Alexander S Ecker, and Matthias Bethge. A neural algorithm of artistic style. *arXiv preprint arXiv:1508.06576*, 2015.
- R. Gonzales et al. A retrieval augmented approach for text-to-music generation. In *NLP4MuSA Workshop*, 2024.
- Q. Huang et al. Make-an-audio: Text-to-audio generation with prompt enhancement. *arXiv preprint arXiv:2308.11752*, 2023.
- H. Jin et al. Multimodal retrieval augmented generation framework for visually-rich document understanding. *Springer Nature Applied Sciences*, 2025.
- Y. Jin et al. Multimodal retrieval-augmented generation for instruction manual understanding. In *Stanford CS231N Class Projects*, 2024.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems (NeurIPS)*, 33:9459–9474, 2020.
- B. Liang et al. Text-to-audio generation using instruction-guided latent diffusion. *arXiv preprint arXiv:2404.09956*, 2024.
- Haohe Liu, Zehua Chen, Yi Yuan, Xinhao Mei, Xubo Liu, Danilo Mandic, Wenwu Wang, and Mark D Plumbley. Audioldm: Text-to-audio generation with latent diffusion models. In *International Conference on Machine Learning (ICML)*, 2023.
- Haohe Liu, Yi Yuan, Xubo Liu, Xinhao Mei, Qiuqiang Kong, Qiao Tian, Yuping Wang, Wenwu Wang, Yuxuan Wang, and Mark D Plumbley. Audioldm 2: Learning holistic audio generation with self-supervised pretraining. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 32:2871–2883, 2024a.
- Pei Liu, Xin Liu, Ruoyu Yao, Junming Liu, Siyuan Meng, Ding Wang, and Jun Ma. Hm-rag: Hierarchical multi-agent multimodal retrieval augmented generation. In *ACM International Conference on Multimedia*, 2025.
- Y. Liu et al. Ddsp-sfx: Acoustically-guided sound effects generation with ddsp. 2024b.
- Marco A Martínez Ramírez and Joshua D Reiss. Deep learning for automatic mixing. In *Audio Engineering Society Convention*, 2020.
- McWec et al. Trends in audio texture analysis, synthesis, and applications. *ResearchGate*, 2023.
- Eloi Moliner, Marco A Martínez-Ramírez, Junghyun Koo, Wei-Hsiang Liao, Kin Wai Cheuk, Joan Serrà, Vesa Välimäki, and Yuki Mitsufuji. Automatic music mixing using a generative model of effect embeddings. *arXiv preprint arXiv:2511.08040*, 2025.
- D. Niizumi et al. M2d-clap: Masked modeling duo meets clap for learning general-purpose audio-language representations. 2024.
- C. Peladeau et al. Blind estimation of audio effects using an auto-encoder architecture. *arXiv preprint arXiv:2310.11781*, 2024.
- M. Schedl et al. Music information retrieval: Recent developments and future challenges. *Foundations and Trends in Information Retrieval*, 8(2-3):127–261, 2014.

- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- N. Srivatsan et al. Retrieval guided music captioning via multimodal prefixes. In *International Joint Conference on Artificial Intelligence (IJCAI)*, 2024.
- Ryan Stables, Sean Enderby, Brecht De Man, György Fazekas, and Joshua D Reiss. Safe: A system for the extraction and retrieval of semantic audio descriptors. In *Proceedings of the AES 53rd International Conference on Semantic Audio*, 2014.
- Christian J Steinmetz, Nicholas J Bryan, and Joshua D Reiss. Automatic multitrack mixing with a differentiable mixing console of neural audio effects. In *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2022.
- Yannis Stylianou, Georg Kappler, and George Tzanetakis. Audio style transfer. In *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2015.
- Renxi Sun et al. Toolgen: Unified tool retrieval and calling via generation. *arXiv preprint arXiv:2410.03439*, 2024.
- Lavinia Verzotto et al. Audio representations for deep learning in sound synthesis. *arXiv preprint arXiv:2201.02490*, 2022.
- Apoorv Vyas, Bowen Shi, Matthew Le, Andros Tjandra, Yi-Chiao Wu, Baishan Guo, Jiemin Zhang, Xinyue Zhang, Robert Adkins, William Ngan, et al. Audiobox: Unified audio generation with natural language prompts. *arXiv preprint arXiv:2312.15821*, 2023.
- S. Wang et al. Audiolcm: Efficient and high-quality text-to-audio generation via latent consistency models. 2024.
- B. Weck et al. Wikimute: A web-sourced dataset of semantic descriptions of music. *arXiv preprint arXiv:2312.09207*, 2023.
- Y. Wu et al. Few-shot prompt learning for audio language models. 2024.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- Michihiro Yasunaga, Armen Aghajanyan, Weijia Shi, Richard James, Jure Leskovec, Percy Liang, Mike Lewis, Luke Zettlemoyer, and Wen-tau Yih. Retrieval-augmented multimodal language modeling. *arXiv preprint arXiv:2211.12561*, 2023.
- H. Zayats et al. Mmkb-rag: A multi-modal knowledge-based retrieval-augmented generation system. *arXiv preprint arXiv:2504.10074*, 2024.